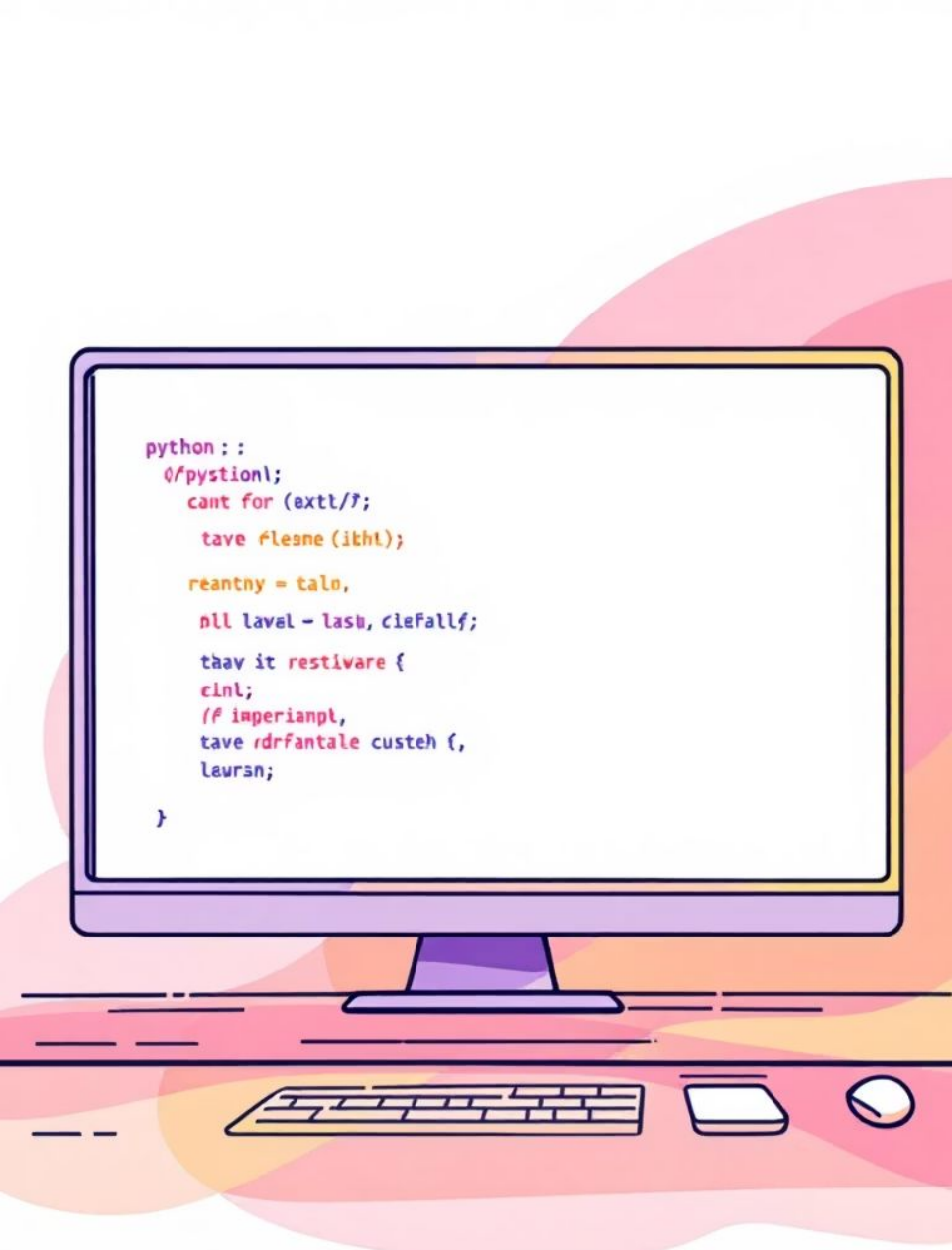


Classes e Objetos em Python



```
python : :  
0/pystionl;  
cant for (extt/f;  
tave flesne (ithl);  
reantny = talo,  
nll lavel - lash, cleFallf;  
thav it restivare {  
cintl;  
/f imperianpl,  
tave rdrfantale custeh {,  
lauran;  
}
```

Objeto com Atributos

Um objeto é uma instância de uma classe, contendo atributos que guardam estado. A classe define como o objeto é criado e quais validações são aplicadas.

Atributos ficam acessíveis via `obj.atributo`.

```
class Polygon:
    def __init__(self, n: int):
        self.n = n

p = Polygon(5)
print(p.n) # 5
```

Construtor com Validação

Criação Consistente

O construtor `__init__` cria e inicializa objetos de forma consistente.

Validação de Entrada

Evita estados inválidos (ex: número de vértices ≤ 0).

Exceções Claras

Se a validação falhar, lance uma exceção com mensagem clara.

```
class Polygon:
    def __init__(self, n: int):
        if n <= 0:
            raise ValueError("número de vértices deve ser maior que zero")
        self.n = n
```

```
p = Polygon(5)
print(p.n)    # 5
# Polygon(0)  # ValueError: número de vértices deve ser maior que zero
```

Especializando Polygon em Rectangle

Polygon

Classe base com `n` vértices

Rectangle

Especialização com 4 vértices e atributos `w`, `h`

Herança permite reutilizar código e especializar comportamento. A relação "é-um" fica explícita: Rectangle **é um** Polygon.

```
class Rectangle(Polygon):  
    def __init__(self, w: float, h: float):  
        super().__init__(4)  
        self.w = w  
        self.h = h
```

```
r = Rectangle(3, 10)  
print(r.n, r.w, r.h) # 4 3 10
```

Interface de Impressão com `__str__`

Polimorfismo em Ação

Em Python, controla-se `print(obj)` implementando `__str__`.

Cada classe define sua própria representação textual. O mesmo `print()` exibe coisas diferentes conforme o tipo do objeto.

```
class Polygon:
    def __init__(self, n: int):
        self.n = n

    def __str__(self) -> str:
        return f"{self.n}"

class Rectangle(Polygon):
    def __init__(self, w: float, h: float):
        super().__init__(4)
        self.w = w
        self.h = h

    def __str__(self) -> str:
        return f"{self.w}, {self.h}"

p = Polygon(5)
r = Rectangle(3, 10)
print(p) # 5
print(r) # 3, 10
```

Polimorfismo

Mesmo Comando

O `print()` funciona para diferentes tipos de objetos.

Resultados Diferentes

Cada classe define seu próprio `__str__`, produzindo saídas específicas.

Comportamento Dependente

Comportamento dependente do tipo é chamado de **polimorfismo**.

```
p = Polygon(6)
r = Rectangle(2, 4)
print(p) # usa Polygon.__str__()
print(r) # usa Rectangle.__str__()
```

Criando a Interface area()

Uma interface é uma função que funciona para vários tipos de objetos. Em Python, isso é feito definindo um método com o mesmo nome nas classes.

Cada classe implementa sua própria versão de `area()`.

```
class Polygon:
    def __init__(self, n: int):
        self.n = n

    def __str__(self) -> str:
        return f"{self.n}"

    def area(self) -> float:
        return 0.0 # área genérica desconhecida
```

Mantemos `__init__` e `__str__` e acrescentamos `area()`.

Método Padrão e Método Específico

Método Padrão

Trata casos genéricos na classe base

Sobrescrita

Classes especializadas sobrescrevem o método

Comportamento Próprio

Cada classe implementa sua lógica específica

```
class Rectangle(Polygon):  
    def __init__(self, w: float, h: float):  
        super().__init__(4)  
        self.w = w  
        self.h = h  
  
    def __str__(self) -> str:  
        return f'{self.w}, {self.h}'  
  
    def area(self) -> float:  
        return self.w * self.h
```

Quando chamamos `r.area()`, Python procura o método primeiro em `Rectangle` e, se não encontrar, em `Polygon`. Esse é o mecanismo de sobrescrita.

Descobrendo Interfaces Disponíveis

Inspeção de Objetos

Em Python, podemos inspecionar um objeto para ver seus métodos disponíveis.

Função `dir()`

`dir()` lista todos os atributos e métodos do objeto, inclusive os especiais (`__init__`, `__str__`...).

Descoberta de Interfaces

Ajuda a descobrir quais "interfaces" um objeto oferece.

```
p = Polygon(5)
r = Rectangle(3, 10)
[m for m in dir(r) if not m.startswith("_")]
# ['area', 'h', 'n', 'w']
```

Usando os Objetos na Prática

Criação e Uso

- Criamos objetos de tipos diferentes
- Chamamos os mesmos métodos neles
- Cada classe responde de forma adequada

```
p = Polygon(5)
r = Rectangle(3, 10)

print(p) # 5
print(r) # 3, 10
print(p.area()) # 0.0
print(r.area()) # 30
```

Estendendo o suporte a polígonos

Square

Deve herdar de Rectangle com lado igual

```
import math

class Square(Rectangle):
    def __init__(self, lado: float):
        super().__init__(lado, lado)
```

Hexagon

Deve herdar de Polygon com área própria

```
class Hexagon(Polygon):
    def __init__(self, lado: float):
        super().__init__(6)
        self.lado = lado

    def area(self) -> float:
        return (3 * math.sqrt(3) / 2) * self.lado ** 2
```

```
s = Square(4)
h = Hexagon(2)
print(s.area()) # 16
print(h.area()) # 10.392304845413264
```

O Que Vem a Seguir

Fundamentos Dominados

Você já domina classes e objetos em Python.

Construtores e Herança

Sabe criar construtores, herança e polimorfismo.

Próximos Passos

Esses conceitos são base para trabalhar com dados reais.

Referências

Think Python

Downey, A. *Think Python: How to Think Like a Computer Scientist*. O'Reilly Media.

Introdução essencial à programação e ao pensamento computacional com Python.

Python Data Science Handbook

VanderPlas, J. *Python Data Science Handbook*. O'Reilly Media.

Guia completo das ferramentas essenciais para trabalhar com dados em Python.

Data Science from Scratch

Grus, J. *Data Science from Scratch*. O'Reilly Media.

Aprenda ciência de dados construindo algoritmos e ferramentas do zero com Python.